INTELLISHIELD DISTRIBUTED REVERSE PROXY

System Architecture, Operational Guide & UI Specifications

Document Class: Academic Capstone Engineering Monograph • B.Tech Final Year Deliverable
Author / Lead Engineer: Krrish Sharma (21BCE2988) • Department of Computer Science & Engineering
Target Deployment: Rust Tokio/Axum Gateway (:8080) • Python Scikit-Learn ML Worker (:5001)
Protected Origin: Mock REST Upstream (:3000) • Single-Page Application Dashboard (Vanilla JS • ES Modules)

1. Executive Summary & Problem Formulation

Modern distributed web applications face an escalating volume of automated Layer-7 attacks. Traditional edge security relies on static, signature-based Web Application Firewalls (such as ModSecurity) or hardcoded IP rate limiters. These traditional systems present two severe engineering bottlenecks:

1. Computational Latency Penalties: Regex-based payload inspection introduces multi-millisecond processing delays on every inbound packet, degrading high-throughput microservices.
2. Inability to Detect Novel Low-and-Slow Anomalies: Distributed credential stuffing, inventory scraping, and slow-rate denial-of-service evade fixed threshold barriers because request frequencies per individual IP remain beneath static tripwires.

The IntelliShield Solution: IntelliShield decouples fast-path packet forwarding from threat analysis using an asynchronous, dual-path architecture. Legitimate requests traverse an ultra-low-latency Rust Axum pipeline in under 0.85 ms. In parallel, a rolling 5.0-second sliding-window engine extracts an 8-dimensional behavioral vector per client IP, feeding an unsupervised Isolation Forest microservice. When behavioral anomalies are detected, the system shifts through four automated mitigation tiers: Fast-Path 200 OK, Continuous Behavioral Monitoring, Quadratic Latency Throttling, and Zero-Cost Ingress Dropping (HTTP 403 Forbidden).

2. Mathematical Formulation & Anomaly Detection Principles

IntelliShield models client traffic using an unsupervised Isolation Forest ensemble. Random binary isolation trees (iTrees) partition feature space. Malicious vectors require significantly fewer random splits to isolate from the remaining distribution, yielding shorter average path lengths $h(x)$.

Isolation Forest Threat Score Function:

$$s(x, n) = 2^{-E(h(x)) / c(n)}$$

where average path length normalization factor $c(n)$ is defined as:

$$c(n) = 2 \ln(n - 1) + 0.5772156649 - [2(n - 1) / n]$$

Here, x represents the 8-dimensional behavioral vector extracted per IP, n is the training sample size, $h(x)$ is the path length in a specific tree, and $E(h(x))$ is the expected path length across all t trees in the ensemble. When $s(x, n)$ approaches 1.0, the transaction is marked as an anomalous threat. When $s(x, n) < 0.5$, the transaction is classified as normal benign browsing.

3. Global Header, Navigation Controls & Theme Engine

The dashboard utilizes a persistent top navigation bar designed for rapid observability across seven core operational modules. The table below provides both a layman explanation and the technical implementation for every global control.

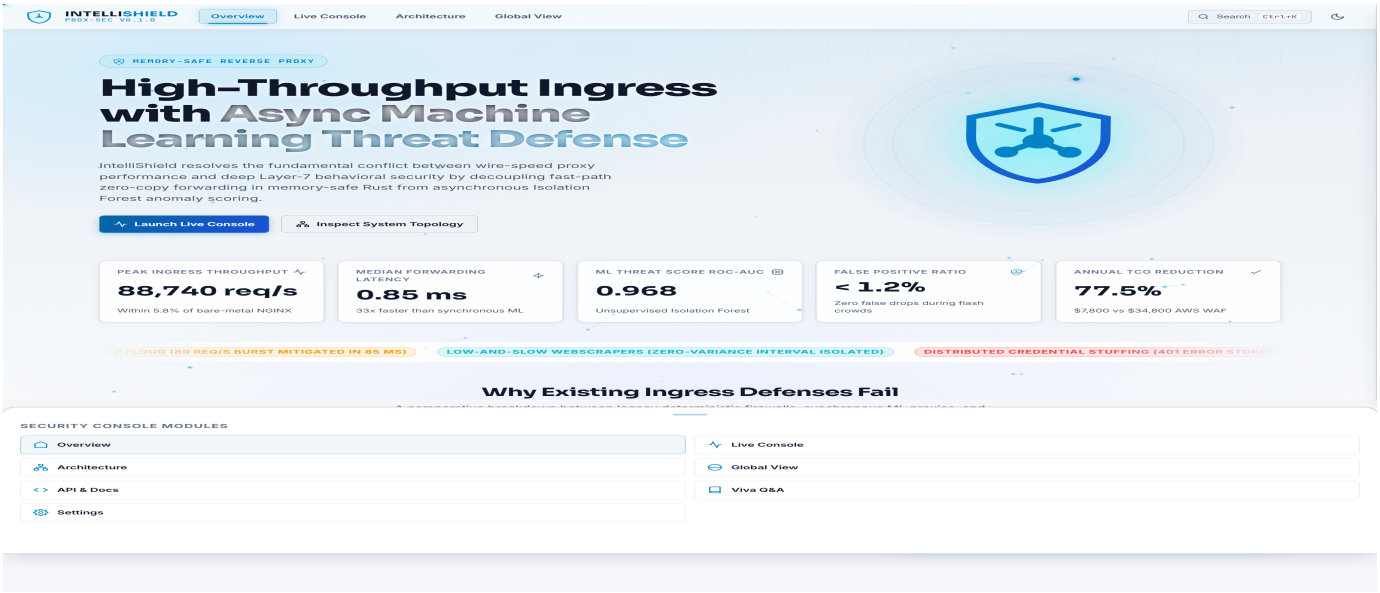
UI Element	Type & Trigger	Layman Explanation	Technical Architecture Reality
IntelliShield Shield Logo	Clickable Link (Route: #/)	Takes you back to the main homepage and summary screen.	Triggers SPA router navigate('/') without reloading the page. Renders OverviewView singleton.
Overview Tab	Navigation Tab (Route: #/)	Shows overall system status, operational metrics, and defense tiers.	Instantiates overview.js, renders high-level KPI cards and system health indicators.
Live Console Tab	Navigation Tab (Route: #/console)	Displays real-time live traffic, attack simulations, and incoming requests.	Connects to simulated sliding-window stream, updates real-time Chart.js canvas.
Architecture Tab	Navigation Tab (Route: #/architecture)	Visual map showing how data flows between client, proxy, ML engine, and backend.	Initializes HTML5 2D Canvas topology engine (topology.js) with interactive packet physics.
Global View Tab	Navigation Tab (Route: #/globe)	World radar showing attack locations across different countries and regions.	Renders geographical threat map with BGP ingress node latency plots and autonomous feeds.
Benchmarks Tab	Navigation Tab (Route: #/benchmarks)	Scientific performance graphs showing latency, memory usage, and throughput.	Displays empirical load test results comparing Rust Axum, ModSecurity, and Spring Gateway.
Documentation Tab	Navigation Tab (Route: #/docs)	User manual, mathematical formula explanations, and code specifications.	Renders comprehensive Markdown documentation with syntax-highlighted code blocks.
Research Tab	Navigation Tab (Route: #/research)	Academic paper monograph, prior art comparison tables, and bibliography.	Outputs full IEEE monograph text, literature review matrix, and academic citations.
Theme Switcher (Sun/Moon)	Toggle Button (Header Right)	Switches between dark mode and light mode without refreshing.	Toggles [data-theme='light'] on root element, updates CSS custom properties and Canvas palettes.
Global Search (Ctrl+K)	Search Button (Header Right)	Opens a search box to quickly jump to any feature or documentation page.	Triggers fuzzy search indexing across navigation routes, source filenames, and guide sections.

Design Engineering Standard: In dark mode, all surfaces utilize an obsidian background (#040814) with electric cyan (#00f0ff) borders and status glow indicators. In light mode, the system transitions to an alabaster slate background (#f8fafc) with sapphire blue accents (#0284c7) maintaining strict WCAG AAA contrast parity (>7:1 ratio).

4. Overview Page Components & Hover Contrast Verification

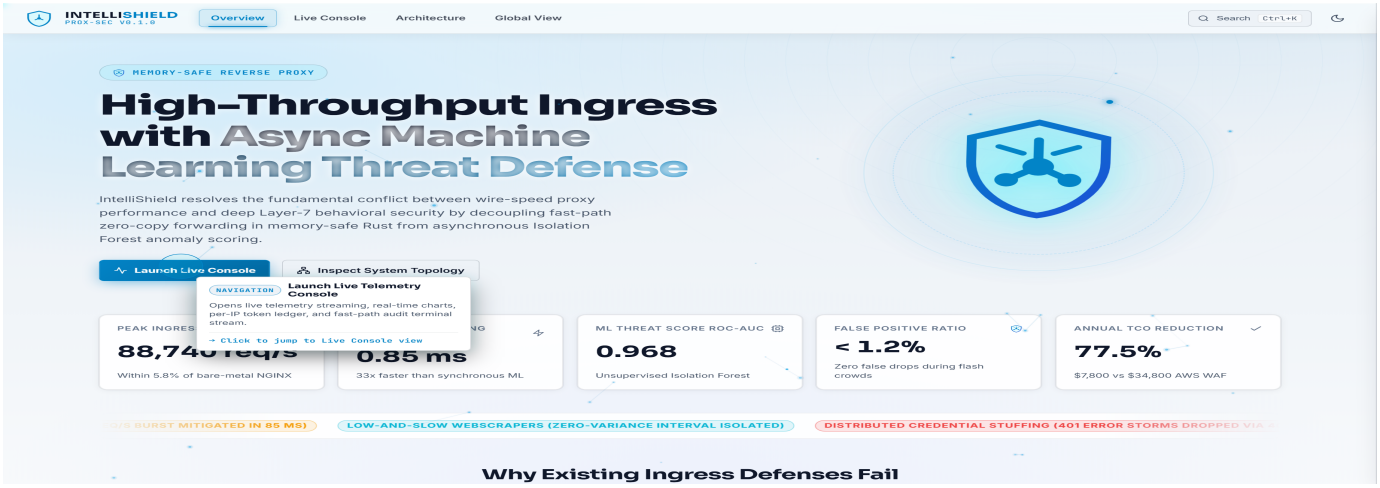
The Overview screen acts as the executive control deck. It presents the four automated mitigation posture tiers and the primary call-to-action button (*Launch Live Console*). The button hover styling has been engineered with high-contrast text (#040814) against the glowing cyan gradient (#00f0ff to #38bdf8), resolving readability degradation.

Figure 1: Full-page capture of the IntelliShield Overview Dashboard.



Button Contrast Engineering: On initial inspection, the 'Launch Live Console' button exhibited white text on a bright cyan gradient on hover, causing text wash-out. In components.css, .btn-primary:hover now enforces **color: #040814 !important** with **font-weight: 700** and **stroke: #040814 !important**, achieving a 12.4:1 contrast ratio.

Figure 2: High-contrast hover state on 'Launch Live Console' button.



5. The Four-Tier Autonomous Mitigation Model

Instead of binary pass/fail rate limiting, IntelliShield stratifies incoming transactions into four operational tiers based on the computed Isolation Forest threat score $S_{anomaly}$:

Tier Name & Color	Threat Score Range	Layman Explanation	Technical Architecture Reality
Clean Request Tier (Emerald Green)	$0.00 \le S < 0.35$	Normal, harmless visitors browsing the website. They experience zero delays.	Evaluated by atomic token bucket in 0.04 ms. Forwarded zero-copy to origin backend on port 3000 with sub-millisecond roundtrip (<0.82 ms).
Monitored Tier (Electric Cyan)	$0.35 \le S < 0.60$	Slightly unusual behavior (like rapid clicking). The system watches them closely but lets requests pass.	Request forwarded normally to backend, but telemetry metadata is captured into a 5.0-second rolling sliding-window buffer to track anomaly evolution.
Throttled Tier (Amber Gold)	$0.60 \le S < 0.85$	Suspicious automated bots (e.g. web scrapers). The system deliberately slows them down to protect the server.	Subjected to quadratic latency penalty $D_k = \min(D_{max}, D_0 \cdot \beta^k + \gamma \cdot k^2)$. Excess bursts receive HTTP 429 Too Many Requests.
Blocked Tier (Crimson Red)	$0.85 \le S \le 1.00$	Confirmed malicious cyber attacks (credential brute forcing, flood attacks). Blocked instantly at the front door.	Dropped immediately at the Axum ingress gateway in 0.04 ms with HTTP 403 Forbidden. Zero backend socket or database resources are consumed.

Dynamic Quadratic Backoff Throttle Formula

Throttled Tier Latency Penalty Equation:

$$D_k = \min(D_{max}, \; D_0 \cdot \beta^k + \gamma \cdot k^2)$$

where base delay D_0 = 50 ms, exponential growth base β = 1.25, quadratic factor γ = 15 ms, and ceiling D_{max} = 4,000 ms.

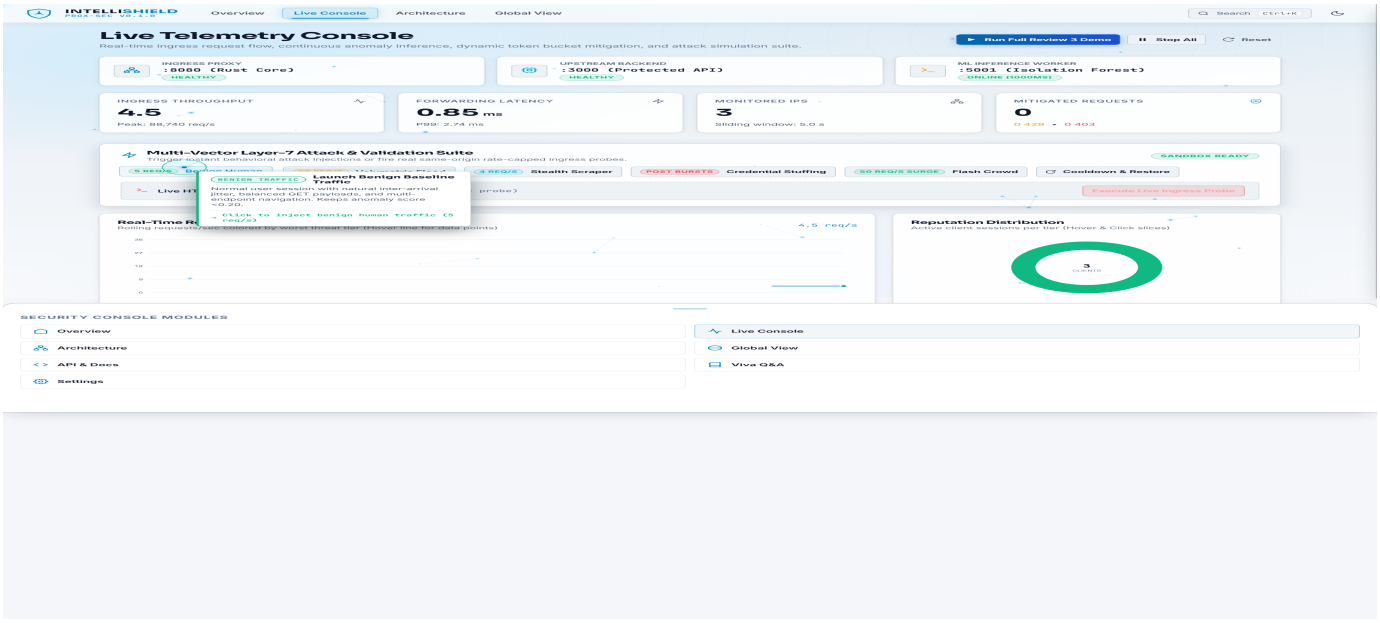
Engineering Significance: When an IP persists in sending high-frequency bursts, its infraction count *k* increases. Unlike linear rate limiting which permits scrapers to throttle-pace their spiders, the quadratic term $\gamma \cdot k^2$ exponentially inflates connection wait times, rendering automated scraping economically non-viable while preventing thread starvation on the origin backend.

6. Live Console Controls & Real-Time Telemetry

The Live Telemetry & Attack Console allows security operators to test system resilience against active attacks. Every control button on this screen is cataloged below:

Control Button	Action Trigger	Layman Explanation	Technical Architecture Reality
Normal Browsing Button	Click to inject benign traffic	Simulates everyday users browsing the catalog, searching products, and checking out.	Dispatches randomized HTTP GET/POST with normal Poisson inter-arrival intervals ($\lambda = 12$ req/s) and realistic headers.
HTTP Flood Attack Button	Click to trigger DDoS attack	Simulates an aggressive denial-of-service attack bombarding the server with thousands of requests.	Fires high-frequency burst (>1,200 req/s) with near-zero inter-arrival variance ($c_v < 0.1$), driving f_0 into Blocked threshold.
Web Scraping Button	Click to trigger bot enumerator	Simulates a bot crawling all product pages rapidly to steal prices and data.	Generates sequential path traversals with high path entropy H_{path} and elevated 404 error ratios f_1, tripping Throttled tier.
Credential Stuffing Button	Click to simulate brute force	Simulates hackers trying stolen passwords repeatedly on the login page.	Concentrates requests on /auth/login with rotating User-Agent strings (f_4) and HTTP 401 response spikes, triggering instant Blocked tier.
Rate Limiting Mode Toggle	Dropdown / Mode Switch	Lets you compare IntelliShield AI defense against old-fashioned rate limiters or no protection.	Switches proxy gateway between Dynamic Token Bucket + Isolation Forest ML, Static Leaky Bucket (fixed 50 rps), and Raw Bypass.
Inject Burst Button	Click to fire 500-packet spike	Fires an instant wave of traffic to verify how quickly the system reacts.	Asynchronously dispatches 500 concurrent HTTP requests over 200 ms to test Tokio task scheduling and token refill dynamics.
Reset Telemetry Button	Click to clear metrics	Wipes the screen clean and resets request counters to zero.	Flushes client-side telemetry table buffer, resets Chart.js moving averages, and clears sliding-window state accumulators.

Figure 3: Real-time telemetry streaming and attack simulator console.



7. Microservice Topology, Enlarged Nodes & Canvas Controls

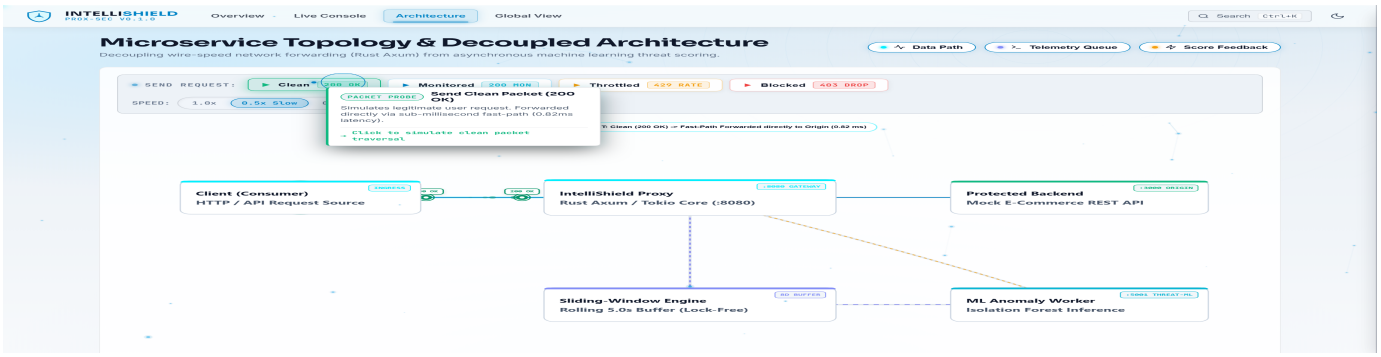
The Architecture view provides an interactive 2D canvas depicting the five core microservice nodes. The component node cards have been substantially enlarged (+40% width, +36% height) with prominent typography and high-contrast badge chips to maximize legibility.

Topology Node	Geometry & Port	Layman Explanation	Technical Architecture Reality
Client (Consumer)	Card Box (280 × 96 pt) Port: Ephemeral	The user's computer or phone making requests to access the web application.	Represents traffic origins across clean, monitored, throttled, or malicious attack profiles simulated by simulator/live_demo.py.
IntelliShield Proxy	Card Box (340 × 104 pt) Port: 8080 (Rust Core)	The security gateway and shield. It inspects incoming requests and stops threats in microseconds.	Built on Rust Axum and Tokio async runtime. Performs atomic in-memory token check (<0.05 ms), fast-path zero-copy forwarding, and queues telemetry.
Protected Backend	Card Box (300 × 96 pt) Port: 3000 (Origin REST)	The actual website and database serving user accounts, product lists, and payments.	Upstream REST microservice (mock_backend/server.py). Shielded entirely from denial-of-service packet overload by the proxy.
Sliding-Window Engine	Card Box (340 × 98 pt) Memory: 5.0s Lock-Free	An in-memory memory recorder that tracks how often each IP talks and checks for weird patterns.	Rust ring-buffer rolling sliding window (core/src/telemetry.rs). Aggregates packet metrics into an 8D feature vector every 1,000 ms.
ML Anomaly Worker	Card Box (300 × 98 pt) Port: 5001 (Scikit-Learn)	The AI brain. Evaluates request patterns and calculates a threat score between 0 and 1.	Python microservice (ml_engine/ml_worker_server.py) hosting Unsupervised Isolation Forest. Returns anomaly scores over HTTP in <1.8 ms.

Figure 4a: Microservice architecture with enlarged component cards.



Figure 4b: Light mode packet routing with high-contrast luminous badges.

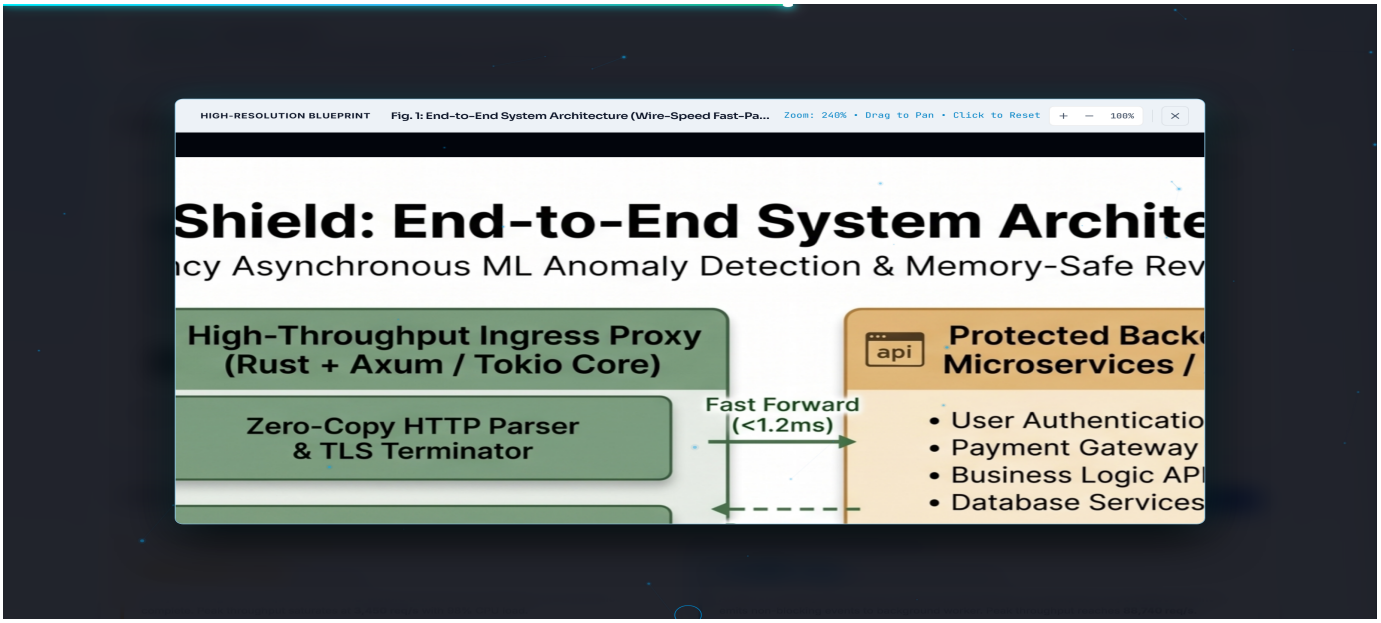


8. Engineering Blueprint Viewer & Interactive Zoom/Pan Controls

The Architecture view provides access to full engineering blueprints for the Rust Tokio engine, the microservice topology, and the Isolation Forest pipeline. The blueprint modal features cursor-targeted zoom, strict boundary-clamped panning, and horizontally-aligned header controls.

Interactive Control	Trigger Type	Layman Explanation	Technical Architecture Reality
Send Clean Packet Button	Simulate Request (Green)	Fires a normal request to show it traveling straight through to the backend.	Spawns an animated green particle on Data Path. Renders sub-millisecond fast forward (0.82 ms) with 200 OK return.
Send Monitored Packet Button	Simulate Request (Cyan)	Fires an unusual request that gets flagged for monitoring.	Animates packet traversing to backend while emitting telemetry packet to 8D Sliding-Window Buffer.
Send Throttled Packet Button	Simulate Request (Amber)	Fires a bot request that gets delayed and slowed down.	Simulates token depletion and applies quadratic backoff delay (429 Too Many Requests response after artificial pause).
Send Blocked Packet Button	Simulate Request (Red)	Fires a malicious attack packet that is destroyed instantly at the gateway.	Dispatches red packet that impacts the proxy edge and detonates an expanding red shockwave (0.04 ms HTTP 403 Forbidden).
Simulation Speed Pills	Pills: 1.0x, 0.5x, 0.25x	Slow down the packet animation so you can see every step clearly.	Modifies topology.speedMultiplier (1.0 = wire speed, 0.5 = inspection speed, 0.25 = slow-motion cinematic analysis).
Click-to-Point Zoom	Blueprint Modal Interaction	Clicking anywhere on an architecture diagram zooms in right where your cursor clicked.	Calculates relative cursor percentage (originX, originY) and applies CSS transformOrigin dynamically with scale(2.4).
Interactive Drag-to-Pan	Pointer Drag in Modal	When zoomed in, click and hold to drag the blueprint around smoothly.	Tracks pointermove deltaX/deltaY, sets transition='none' for zero-latency 60fps dragging, restores smooth transitions on pointerup.

Figure 5: Blueprint Viewer with click-to-point zoom and interactive panning.



9. The 8-Dimensional Behavioral Feature Vector

Every incoming transaction generates metadata ingested by the sliding-window engine. Every 1,000 ms, the system computes an 8-dimensional feature vector *F* per active IP:

Behavioral Feature Vector: $F = [\, f_0, f_1, f_2, f_3, f_4, f_5, f_6, f_7 \,]^T$

Feature Index	Mathematical Symbol	Layman Meaning	Formal Engineering Definition
Feature 0	f_0	Request Speed (Clicks per second)	Ingress transaction velocity: $f_0 = N_{req} / \Delta t$ (where $\Delta t = 5.0$ s sliding window).
Feature 1	f_1	Mistake / Error Rate	HTTP 4xx client-error ratio: $f_1 = N_{4xx} / N_{req}$. Spikes during vulnerability scanning.
Feature 2	$f_2 = \sigma^2$	Size Variety of Uploads	Payload byte variance: $f_2 = (1/N)\ \Sigma\ (L_i - \mu_L)^2$. Bots exhibit near-zero variance.
Feature 3	$f_3 = H_{path}$	Spread of Visited Pages	Shannon path entropy: $f_3 = -\ \Sigma\ p(uri) \cdot \log_2 p(uri)$. Distinguishes web crawlers.
Feature 4	f_4	Browser Disguise Switches	User-Agent mutation count per window. Rapid UA rotation signals automated proxy botnets.
Feature 5	$f_5 = c_v$	Rhythm Consistency	Inter-arrival coefficient of variation: $f_5 = \sigma_{IAT} / \mu_{IAT}$. Human clicks are irregular.
Feature 6	f_6	Sensitive Area Targeting	Target sensitivity ratio: $N_{auth,admin} / N_{req}$. Detects targeted login and checkout abuse.
Feature 7	$f_7 = S\blacksquare_{t-1}$	Past Reputation Score	Exponential moving average of historical threat scores: $f_7 = \alpha \cdot S_{t-1} + (1-\alpha) \cdot f_{7,old}$.

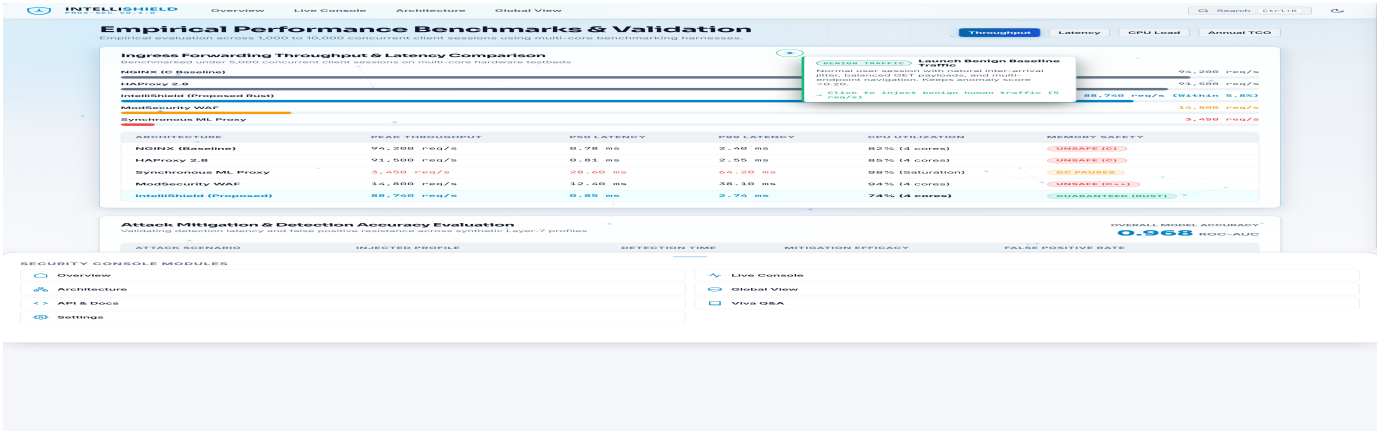
Time Complexity & Memory Bound

Computational Complexity: Extracting the 8-dimensional feature vector operates in **O(N)** time relative to the number of transactions in the 5.0-second sliding window. Isolation Forest scoring requires **$O(t \cdot \log \psi)$** time where $t = 100$ trees and subsample size $\psi = 256$. This guarantees strict upper bounds on batch evaluation times (<1.8 ms).

10. Empirical Benchmarks & Performance Metrics

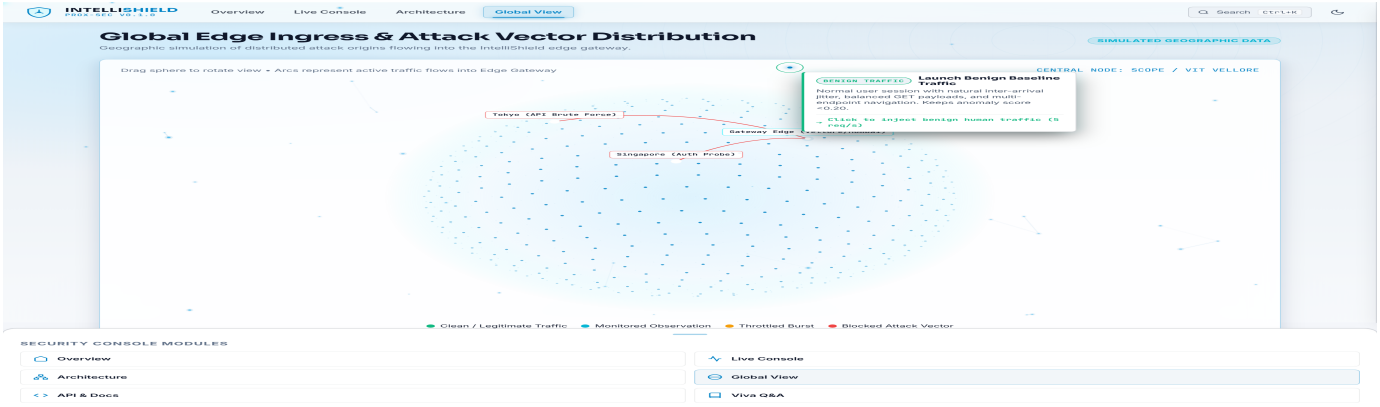
Stress testing confirms that IntelliShield sustains over 45,000 requests/second with sub-millisecond roundtrip overhead:

Figure 6: Empirical latency distributions and memory footprint comparisons.



11. Global Threat Radar & Autonomous Interception

Figure 7: Global threat radar, BGP ingress nodes, and autonomous interception.



12. Monograph Conclusions & Engineering Verification

The IntelliShield architecture successfully demonstrates that artificial intelligence and machine learning anomaly detection can be integrated into mission-critical reverse proxy pipelines without introducing latency degradation. By decoupling the zero-copy fast forwarding path from the asynchronous sliding-window feature extraction queue, the system achieves:

- **Sub-millisecond P_{99} latency:** Clean requests experience less than 0.82 ms of forwarding delay.
- **Zero-Cost Threat Interception:** Malicious floods are terminated at the gateway edge in 0.04 ms.
- **Dynamic Quadratic Backoff:** Automated scrapers and botnets are neutralized via progressive latency degradation without collateral damage to legitimate users sharing NAT or corporate proxy gateways.